Measurements and Metrics

- Software measurement **quantifies** the characteristics of a software product or the software process.
- Software measurement process includes
  - **Formulation**: The **derivation** of software measures and metrics appropriate for the representation of the software that is being considered.
  - **Collection**: The mechanism used to **accumulate** data required to derive the formulated metrics.
  - **Analysis**: The **computation** of metrics and the application of mathematical tools.
  - **Interpretation**: The evaluation of metrics resulting in **insight** into the quality of the representation.
  - **Feedback**: **Recommendation** derived from the interpretation of product metrics transmitted to the software team.

Software Quality Metrics

- Software quality metrics are a subset of software metrics that focus on the quality aspects of the product, process, and project.
- Metrics have been proposed for many quality attributes
- We may also define own metrics

*Even if a metric is not a measurement (measurement is the act of collecting data, while metrics are derived from one or more measures to track a process or assess a particular goal), often the two terms are used as synonyms (i.e., 软件度量).

Software Qualities

- Scalability
- Security
- Extensibility
- Documentation
- Performance
- Consistency
- Portability
- Installability
- Maintainability
- Functionality (e.g., data integrity)
- Availability
- Ease of use

- On-time release
- Development speed
- Meeting efficiency
- Conformance to processes
- Time spent on rework
- Reliability of predictions
- Fairness in decision making

- Measure time, costs, actions, resources, and quality of work packages; compare with predictions
- Use information from issue trackers, communication networks, team structures, etc…

Types of Metrics

Product Metrics	Process Metrics	Project Metrics
Describe the characteristics of the product/software, e.g., size, complexity, performance, etc.	Used for describing and improving software development & maintenance process, e.g., # bugs found, bug fixing time, # features added	Describe the project characteristics and execution, e.g., # of developers, money spent, time spent

产品 过程 项目

Measuring the Complexity of A System

Lines of Code

- LoC is easy to measure (`> wc -l command`)
- All software products produce LoC

Potential problems with the LoC metric?

LOC	projects
450	Expression Evaluator
2,000	Sudoku
100,000	Apache Maven
500,000	Git
3,000,000	MySQL
15,000,000	gcc
50,000.000	Windows 10
2,000,000,000	Google (MonoRepo)

Normalizing

```
for (i = 0; i < 100; i += 1) printf("hello");
```

```
for (
        i = 0;
        i < 100;
        i += 1
    ) {
        printf("hello");
}
```

- LoC needs to be **normalize**d to be meaningful and comparable
- Ignore comments and empty lines
- Count statement (logical lines) instead of textual lines
- See cLoc (https://github.com/AlDanial/cloc)

应只考虑有意义的行

Language matters

Higher-level languages are more expressive than lower-level languages. Each line of code says more.

- Assembly code may be 2-3X longer than C code
- C code may be 2-3X longer than Java code
- Java code may be 2-3X longer than ...

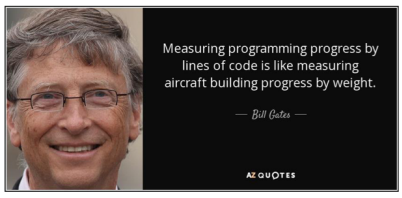


语言也会影响代码行数

- The table shows typical ratios of source statements in several high-level languages to the equivalent code in C.
- A higher ratio means that each line of code in the language listed accomplishes what x lines of code in C accomplish.

https://blog.codinghorror.com/are-all-programming-languages-the-same/

Language	Level Relative to C
C	1
C++	2.5
Fortran	2
Java	2.5
Perl	6
Python	6
Smalltalk	6
MS Visual Basic	4.5



LoC is a valid metric when
- Use within the same programming language
- Code measured use standard, consistent formatting
- Code has been reviewed first

代码行数仅在以下情况下有效
① 用同样语言
② 衡量方式要标准
③ 代码需先审查

Cyclomatic Complexity

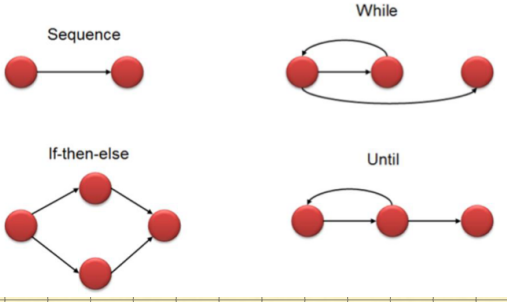
- Cyclomatic complexity (圈复杂度) is a software metric used to indicate the logic complexity of a program.
- It is a quantitative measure of the number of linearly independent paths through a program's source code.
- It was developed by Thomas J. McCabe, Sr. in 1976.

Core idea: the complexity of code depends on the number of decisions in the code (if, while, for)

决策分支的数量.

Control Flow Graph

- Cyclomatic complexity is computed using the control-flow graph (控制流图) of the program
- The nodes of the control flow graph correspond to indivisible groups of commands of a program (e.g., blocks), and a directed edge connects two nodes if the second command might be executed immediately after the first command.



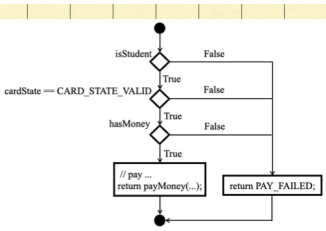
Decisions are **cycles**

Calculate Cyclomatic Complexity

Approach 1: `V (G) = P + 1`
P: Number of branch nodes (e.g., if, for, while, case)

Approach 2: `V(G) = E - N + 2`
E: Number of edges
N: Number of nodes

```
1  if (isStudent) {
2     if (cardState == CARD_STATE_VALID) {
3        if (hasMoney) {
4           // pay ...
5           return payMoney(...);
6        }
7     }
8  }
9  return PAY_FAILED;
```



Cyclomatic complexity = 3 + 1 = 4

Original paper is not clear about how to derive the control flow graph
- Different implementations gives different values for the same code.
- For example, the following code is reported with complexity 2 by the Eclipse Metrics Plugin , with 4 by GMetrics , and with complexity 5 by SonarQube

```
1  int foo (int a, int b) {
2      if (a > 17 && b < 42 && a + b < 55) {
3          return 1;
4      }
5      return 2;
6  }
```

https://www.cqse.eu/en/news/blog/mccabe-cyclomatic-complexity/

控制流图的生成并不清晰.

Interpreting Cyclomatic Complexity

- Cyclomatic complexity is the number of independent paths through the procedure
- Gives an upper bound on the number of tests necessary to execute every edge of control graph
- So, in the context of testing, cyclomatic complexity can be used to estimate the required effort for writing tests.

Cyclomatic complexity	Code	Testability	Maintenance cost
1-10	Structured and well written	High	Low
10-20	Complex	Medium	Medium
20-40	Very complex	Low	High
>40	Unreadable	Very low	Very high

能覆盖所有分支的test数.

Pitfalls on Cyclomatic Complexity

High cyclomatic complexity means poor readability?
- The code got the cyclomatic complexity of 14
- Seems readable

```
String getMonthName (int month) {
    switch (month) {
        case 0: return "January";
        case 1: return "February";
        case 2: return "March";
        case 3: return "April";
        case 4: return "May";
        case 5: return "June";
        case 6: return "July";
        case 7: return "August";
        case 8: return "September";
        case 9: return "October";
        case 10: return "November";
        case 11: return "December";
        default:
            throw new IllegalArgumentException();
    }
}
```

https://www.cqse.eu/en/news/blog/mccabe-cyclomatic-complexity/

Cyclomatic Complexity 并不能反映可读性和可维护性

```
String getWeight(int i) {
    if (i <= 0) {
        return "no weight";
    }
    if (i < 10) {
        return "light";
    }
    if (i < 20) {
        return "medium";
    }
    if (i < 30) {
        return "heavy";
    }
    return "very heavy";
}
```

```
int sumOfNonPrimes(int limit) {
    int sum = 0;
    OUTER: for (int i = 0; i < limit; ++i) {
        if (i <= 2) {
            continue;
        }
        for (int j = 2; j < i; ++j) {
            if (i % j == 0) {
                continue OUTER;
            }
        }
        sum += i;
    }
    return sum;
}
```

Both have a cyclomatic complexity of 5. Same readability & maintainability?

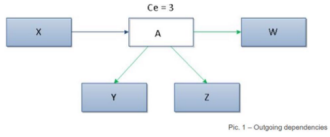
Coupling and Cohesion

- Dependences
  - Call methods, refer to classes, share variables
- Coupling
  - Dependences among modules (bad)
- Cohesion
  - Dependences within modules (good)

Coupling：模块之间的依赖（不好）
Cohesion：模块内部的聚合（好）

Martin's Coupling Metrics

- **Efferent coupling (Ce):** A class's efferent couplings is a measure of how many different classes are used by the specific class.
- Ce measures outgoing dependencies (who do you depend on)
- Measures the vulnerability of the class to changes in other classes that it depends on



Pic. 1 – Outgoing dependencies

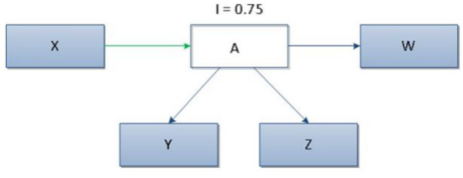
Ce> 20 indicates instability of a package

https://kariera.future-processing.pl/blog/object

即依赖图中节点的出度

- **Instability:** measure the relative susceptibility of class to changes

$$I = \frac{Ce}{Ce + Ca}$$



Pic. 3 – Instability

Ca即入度

- The ones having many outgoing dependencies and not many of incoming ones (instability is close to 1): rather unstable due to the possibility of easy changes to these packages
- The ones having many incoming dependencies and not many of outgoing ones (instability is close to 0): rather more difficult in modifying due to their greater responsibility.

Ce过大，Ca过小：单点失效

Ce过小，Ca过大：难于修改

OO Metrics

OO METRICS ckjm — Chidamber and Kemerer Java Metrics

- Weighted Methods per Class (WMC)
- Depth of Inheritance Tree (DIT)
- Number of Children (NOC)
- Coupling between Object Classes (CBO)
- Response for a Class (RFC)
- Lack of Cohesion in Methods (LCOM)
- ……

Weighted Methods per Class

- This metric is the sum of complexities of methods defined in a class.
- It therefore represents the complexity of a class as a whole
- Possible method complexities
  - 1 (# of methods)
  - LoC
  - # of method calls
  - Cyclomatic complexity

- This measure can be used to indicate the development and maintenance effort for the class.
- The larger the number of methods in a class, the greater the potential impact on children
- Classes with large numbers of methods are more likely to be application specific and less reusable

类中方法越多，对子类影响越大。

Depth of Inheritance Tree

继承的深度

DIT measures the maximum length between a node and the root node in a class hierarchy



- The deeper a class is in the hierarchy, the more methods it inherits and so it is harder to predict its behavior
- The deeper a class is in the hierarchy, the more methods it reuses
- Deeper trees are more complex

Number of Children

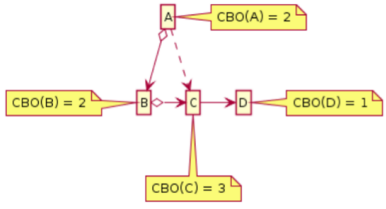
子类数量

子类越多越重要

- NOC indicates the number of immediate subclasses
- A class with a large NOC is probably very important and needs a lot of testing

Coupling Between Object Classes

- CBO represents the number of classes coupled to a given class.
- This coupling can occur through method calls, field accesses, inheritance, arguments, return types, and exceptions.
- CBO doesn't care about the direction of a dependency



Response for a Class

有相同参数列表(或参数列表能自动转的)的方法数量.

- RFC measures the number of different methods that can be executed when an object of that class receives a message (when a method is invoked for that object).
- Ideally, we would want to find for each method of the class, the methods that class will call, and repeat this for each called method, calculating what is called the transitive closure of the method's call graph, which can however be both expensive and quite inaccurate.
- In ckjm, we calculate a rough approximation to the response set by simply inspecting method calls within the class's method bodies.

- If a large number of methods can be invoked in response to a message, testing becomes more complicated
- The more methods that can be invoked from a class, the greater the complexity of the class

Lack of Cohesion in Methods

与类中 sharing fields 无关的方法数

- LCOM counts the sets of methods in a class that are **not related** through the sharing of some of the class's fields.
- Considers all pairs of a class's methods:
  - Q: In some pairs, both methods access at least one common field of the class
  - P: In other pairs, the two methods do not share any common field accesses

$$LCOM = P - Q$$

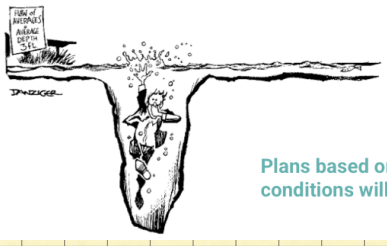
- Cohesiveness of methods is a sign of encapsulation
- Lack of cohesion implies that classes should be split

The Streetlight Effect

- A known observational bias
- People tend to look for something only where it's easiest to do so
  - If you drop your wallet at night, you'll tend to look for it under streetlights
- Are we defining and measuring the right thing(s) to obtain understanding?
- Was only the easy and accessible measured because the important could not be measured?
- What was not measured?

The Flaw of Averages

- A statistician drowns while crossing a river that is only 3-feet deep, on average
- A statistician put his head in the oven and his feet in the freezer so that on average he felt comfortable



The flaw of averages is a set of systematic errors that occurs when people use single numbers (usually averages) to describe uncertain future quantities.

Plans based on the assumption that average conditions will occur are usually wrong

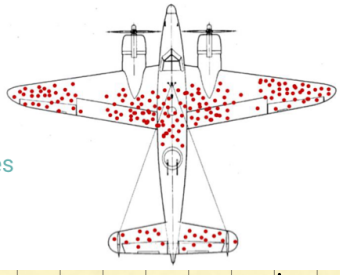
均值不能描述一些极端情况

Survivorship Bias

- Imagine you're in charge of sending airplanes out to fight a war.
- The planes that do come back to base have been hit in the spots indicated by the red dots.
- Where should you add armor to reinforce them?
- In World War II, statistician Abraham Wald recommended that planes be reinforced where **there were no red dots**, assuming that these were the spots that would deal a lethal blow to an airplane.

- Our first instinct is to say "on the red dots!" This is where the planes were hit: let's make these areas stronger

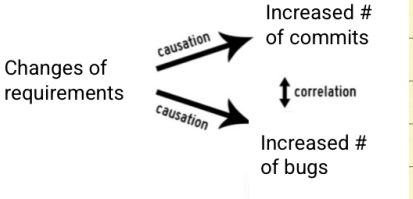
However, we're looking at airplanes that **survived**.



https://www.mcgill.ca/oss/article/general-science/tips-better-thinking-surviving-only-half-story

部分情况对真实情况的遮掩

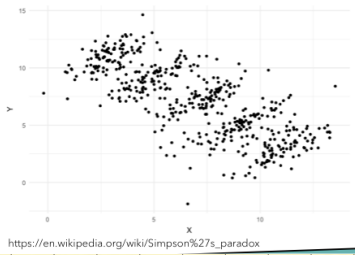
Correlation does not Imply Causation



DEV team concludes that by reducing the number of commits, they can also reduce the number of bugs in the codebase.

Simpson's Paradox

Simpson's paradox is a phenomenon in probability and statistics in which a trend appears in several groups of data but disappears or reverses when the groups are combined.



https://en.wikipedia.org/wiki/Simpson%27s_paradox

在多个群体中都有趋势，但合起来就不一定了

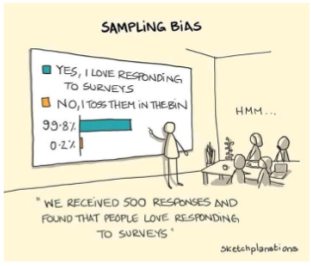
- By analyzing the programming experience and # of bugs produced for each individual developer, the company concludes that less experienced developers are better at writing high-quality code (produce less bugs)
- By analyzing the programming experience and # of bugs produced for each team, the company concludes that teams with more experienced developers are better at writing high-quality code (produce less bugs)

Measurements Validity

- **Construct validity**: are we measuring what we intended to measure?

  To measure "user loyalty", we ask users for their "satisfaction". Is "user satisfaction" the proper measure for "user loyalty"?

- **Internal validity** focuses on the accuracy of the conclusions drawn based on a cause and effect relationship (is the Cause-Effect okay?)



- **External validity**: Can the conclusions be generalizable?

  Our findings are drawn by studying Java code. Can we say the same thing for Python?

Measurements Reliability

- Extend to which a measurement yields similar results when applied multiple times
- Goal is to reduce uncertainty and increase consistency

- Example: measure program performance
  - Time, memory usage
  - Should measure many times
- Law of large numbers (大数定律)
  - As a sample size grows, its mean gets closer to the average of the whole population.
  - Taking multiple measurements to reduce error

Warning

- Most software metrics are controversial
  - Usually only plausibility argument, rarely rigorously validated
  - Cyclomatic complexity was repeatedly refuted and is still used
  - Code size still dominates many metrics
- Metrics can be gamed: you get what you want
- Metrics and measurements are important for decision making.
- Pick or design suitable metrics and use them carefully!